Toronto Bikeshare Demand Forecasting

Esam Uddin
Department of ECE
University of Toronto
esam.uddin@mail.utoronto.ca

Abdul Mohammed
Department of ECE
University of Toronto
abdulrahmanzahid.mohammed@mail.utoronto.ca

Axel Tang
Department of ECE
University of Toronto
axel.tang@mail.utoronto.ca

Abstract

Bike-share systems rely on accurate short-term demand forecasts to support proactive rebalancing. This project predicts hourly Toronto Bike Share pickups using ridership, station, and weather data, comparing a linear regression baseline with Random Forest, XGBoost, and PyTorch MLP models within a shared forecasting pipeline. We also incorporate station clustering and temporal and weather-based features to address sparse, zero-inflated demand, and examine whether improved models can better support truck-based rebalancing and future Bike Angels incentives.

Attestation of Teamwork

Abdul Mohammed: EDA and feature engineering, including zero-inflation analysis, station clustering, weather merging, and train/validation/test split creation.
Axel Tang: Linear regression baseline modeling and evaluation, including baseline metrics and predicted-versus-actual analysis.
Esam Uddin: Improved modeling, including Random Forest, XGBoost, and PyTorch MLP experiments, tuning, and final evaluation.

1 Introduction

Bike-share systems work best when bikes are available at the right place and time, but real demand changes sharply across stations, hours, seasons, and weather conditions. In Toronto Bike Share [2], some areas experience strong commuter peaks while others have many zero-demand hours, making rebalancing difficult to plan in advance. As the system continues to expand with new e-bikes and charging infrastructure [3][4], the need for accurate short-term demand forecasting grows. When shortages are not anticipated, operators must respond reactively after stations become empty or full. Improving short-term demand forecasts would support more proactive truck-based rebalancing and could also help guide future Bike Angels incentives toward locations that are most likely to need bikes [1].

Machine learning is a natural approach for this problem because hourly pickup demand depends on interacting temporal, spatial, and weather signals that are difficult to capture with simple rules alone. Prior work has applied ML to bike-share demand forecasting [6] and used spatiotemporal clustering to improve prediction at scale [7]; this project builds on those ideas within a unified pipeline. In this project, we use ridership, station, and hourly weather data to predict future bike pickups and compare

a linear regression baseline against stronger nonlinear models, including Random Forest, XGBoost, and a PyTorch MLP. To better handle sparse, zero-inflated demand, we incorporate station clustering along with lag, calendar, and weather-based features. Our goal is to evaluate whether these richer models can improve forecasting accuracy enough to make bike-share operations more reliable and efficient.

2 Preliminaries and Problem Formulation

Bike-share demand forecasting can be posed as a supervised regression problem [5]. Given historical ridership records, station metadata, and hourly weather observations, the task is to predict the number of bike pickups that will occur in a given region during a given hour. A key challenge in this dataset is that station-level demand is highly zero-inflated: many station-hour pairs contain no pickups at all, especially in winter or at low-traffic locations. To reduce this sparsity, we group nearby stations using K-means clustering and forecast demand at the cluster-hour level instead of the individual-station level. This produces a more stable target while still preserving spatial structure that is useful for rebalancing decisions.

Formally, for each cluster c and hour t, we predict a target value $y_{c,t}$ equal to the total number of pickups observed in that cluster during that hour. Each training example is represented by temporal, weather, and demand-history features, including hour of day, day of week, month, weekend and holiday indicators, rush-hour flags, temperature, precipitation, wind speed, cluster capacity, and a 24-hour lag feature. The data is split chronologically into train, validation, and test periods so that evaluation reflects true forecasting conditions. Our ultimate goal is to learn models that minimize forecast error on held-out data, measured by MAE, RMSE, and R^2, and to determine whether stronger nonlinear models can outperform a linear regression baseline for proactive bike-share rebalancing.

3 Design

All models share the same pipeline: raw ridership CSVs are cleaned and aggregated, stations are grouped into geographic clusters, features are engineered at the cluster-hour level, and models are trained and evaluated on a chronological split.

Geographic clustering. At the station level, 77.7% of all station-hour pairs contain zero pickups. We apply K-means on station coordinates and aggregate pickups to the cluster level, reducing zero inflation to 37.3%. $K = 8$ was selected by inspecting the elbow in within-cluster inertia over $K \in \{5, \ldots, 30\}$. Figure 1 shows the resulting clusters and zero-inflation reduction.

Features. Each training example is a cluster-hour pair with 12 features: temporal (hour of day, day of week, month, weekend, holiday, rush-hour flag), weather (temperature, precipitation, wind speed, rain indicator), and a 24-hour lag of actual pickups for the same cluster. The lag is computed on the full dataset before splitting to avoid leakage.

Data split. Ridership data was available from January through October 2025, with January and February 2026 as holdout. November and December 2025 are absent from the source and excluded from training to avoid corrupting temporal and lag features with artificial zeros. January 1, 2026 is also dropped from validation since its lag pulls from the empty December 31 entry. Train: January 2 – October 31, 2025; validation: January 2–31, 2026; test: February 2026.

Models. Linear regression serves as the reference baseline with no tuning. Random Forest and XGBoost are selected by grid search on validation RMSE. A PyTorch MLP is trained with early stopping on validation RMSE. All models are evaluated on the February 2026 test set using MAE, RMSE, and R^2.

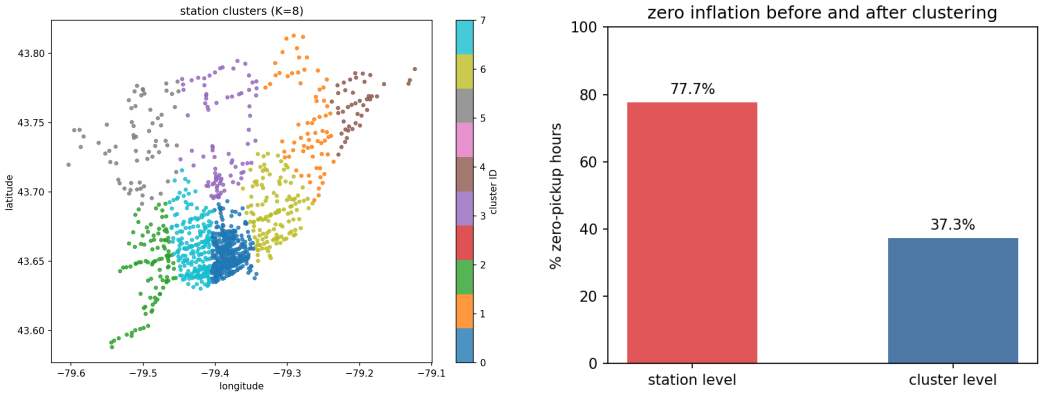


Figure 1: Left: K-means clusters ($K = 8$) on station coordinates. Right: zero-pickup rate before and after cluster-level aggregation.

4 Methodology

In this section we describe the four models used in our forecasting pipeline. All models predict cluster-level hourly pickups from the same feature set. Linear regression drops `cluster_id` and uses 12 features; tree models retain it as a numeric label (13 features); the MLP one-hot encodes it alongside other categorical features.

4.1 Linear Regression. We fit an ordinary least squares model using `sklearn.linear_model.LinearRegression`, which minimizes the squared residual loss with closed-form solution $\hat{\boldsymbol{w}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$. No regularization or hyperparameter tuning is applied. This serves as the reference point to measure whether nonlinear models offer meaningful gains.

4.2 Random Forest. A random forest averages predictions from an ensemble of decision trees, each trained on a bootstrap sample of the data. At each split, a random subset of features is considered, which decorrelates the trees and reduces variance. We use `sklearn.ensemble.RandomForestRegressor` and tune {n_estimators, max_depth, min_samples_split} by grid search on validation RMSE. Best configuration: `max_depth=20`, `n_estimators=200`, `min_samples_split=2`.

4.3 XGBoost. XGBoost [8] builds trees sequentially, where each tree fits the negative gradient of the current ensemble's loss, as shown in Algorithm 1. We use `xgboost.XGBRegressor` with `tree_method=hist` and tune {n_estimators, max_depth, learning_rate} by grid search on validation RMSE. Best configuration: `max_depth=6`, `n_estimators=500`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`.

Algorithm 1 Gradient Boosted Trees

Initialize $F_0(\boldsymbol{x}) = \bar{y}$
for $m = 1, \ldots, M$ **do**
    Compute residuals $r_i = y_i - F_{m-1}(\boldsymbol{x}_i)$
    Fit a regression tree h_m to $\{(\boldsymbol{x}_i, r_i)\}$
    $F_m(\boldsymbol{x}) = F_{m-1}(\boldsymbol{x}) + \eta \cdot h_m(\boldsymbol{x})$
end for
return F_M

4.4 PyTorch MLP. We train a three-layer feedforward network with hidden dimensions [128, 64, 32], ReLU activations, and dropout ($p = 0.2$) after each hidden layer. The output uses Softplus to enforce non-negative predictions. Numeric features are standardized using training set statistics; categorical features (cluster ID, hour, day of week, month) are one-hot encoded. Training uses the Adam optimizer (lr $= 5 \times 10^{-4}$, weight decay $= 10^{-4}$) with MSE loss over up to 15 epochs, and the checkpoint with the lowest validation RMSE is kept. Implementation uses `torch.nn` and `torch.optim`.

5 Numerical Experiments

We evaluate all models on the clustered hourly pickup forecasting task described earlier. The feature set contains 12 inputs: cluster capacity, calendar features, weather features, and a 24-hour lag of demand. We use a chronological split so that training always precedes validation and testing in time. Linear regression serves as the baseline, while Random Forest, XGBoost, and a PyTorch MLP are evaluated as improved models. Model selection is based on validation RMSE, and final results are reported using MAE, RMSE, and R^2 on the test set.

Table 1: Chronological data split used for forecasting experiments.

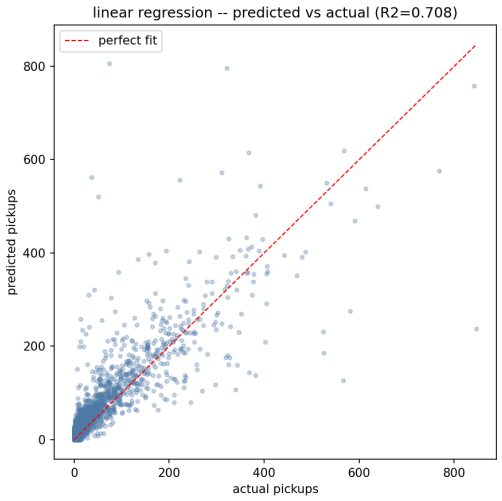
Split	Time period	Samples
Train	2025-01-02 to 2025-10-31	58,176
Validation	2026-01-02 to 2026-01-31	5,760
Test	2026-02-01 to 2026-02-28	5,376

For the tree-based models, we ran small validation-based hyperparameter searches. The best Random Forest used 200 trees with maximum depth 20, while the best XGBoost model used 500 trees with depth 6 and learning rate 0.05. For the PyTorch MLP, we tested three feedforward architectures and selected a network with hidden layers [128, 64, 32], dropout 0.2, learning rate 5×10^{-4}, batch size 512, and best validation performance at epoch 8.

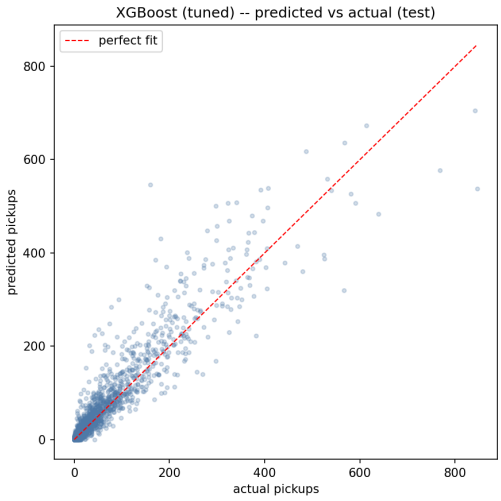
Table 2: Validation and test performance of the forecasting models.

Model	Val RMSE	Test MAE	Test RMSE	Test R^2
Linear Regression	44.52	12.80	34.68	0.678
Random Forest (baseline)	33.95	7.41	23.08	0.857
Random Forest (tuned)	33.97	7.29	22.51	0.864
XGBoost (baseline)	30.78	7.51	22.21	0.868
XGBoost (tuned)	**30.46**	**7.04**	**21.18**	**0.880**
PyTorch MLP	32.89	7.61	23.52	0.852

Table 2 shows that all nonlinear models outperform linear regression by a large margin. The strongest overall model is tuned XGBoost, which achieves the lowest validation RMSE and the best held-out test metrics. Random Forest and the PyTorch MLP also provide major improvements over the linear baseline, but neither surpasses tuned XGBoost on this tabular forecasting setup.



(a) Linear regression baseline



(b) Best tuned tree model (XGBoost)

Figure 2: Predicted-versus-actual test-set performance for the linear baseline and the best tuned tree model. The tuned tree model produces predictions that track the ground truth more closely, especially at moderate and higher demand levels.

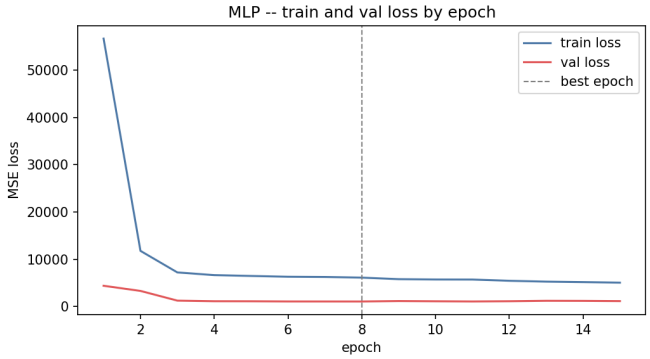


Figure 3: Training and validation loss for the best PyTorch MLP configuration. Validation loss is best at epoch 8, after which gains begin to level off.

6 Discussion

Our experiments were guided by two questions:
1. Can nonlinear models improve bike-share demand forecasting relative to a linear regression baseline?
2. Which improved model performs best on clustered hourly pickup prediction?

The results answer the first question clearly. Linear regression achieved a test MAE of 12.80, RMSE of 34.68, and R^2 of 0.678, but all nonlinear models improved substantially over this baseline. Random Forest reduced test RMSE to 23.08 in its baseline form and 22.51 after tuning, while the PyTorch MLP reached a test RMSE of 23.52 with $R^2 = 0.852$. This suggests that hourly bike-share demand depends on nonlinear interactions between recent demand, calendar effects, and weather.

The second question is also answered consistently by the validation and test results. Tuned XGBoost was the strongest model overall, achieving the best validation RMSE (30.46) and the best test performance, with MAE 7.04, RMSE 21.18, and $R^2 = 0.880$. Compared with Random Forest and the MLP, XGBoost generalized better on this clustered tabular forecasting setup, making it the most effective model in our current pipeline. This is consistent with recent findings that tree-based models tend to outperform deep learning on structured tabular data [9].

7 Conclusions

From this project, we learned that short-term bike-share demand can be forecast effectively using temporal, weather, and lag-based features, especially after clustering stations to reduce zero inflation. The linear regression baseline showed that these engineered inputs already contain useful signal, but the stronger gains came from nonlinear models that can better capture interactions across time, weather, and recent demand history. Among the models we tested, tuned XGBoost delivered the best overall performance on both validation and test data, making it the strongest choice in our current forecasting pipeline.

A natural next step is to strengthen the feature set to further improve our forecasting metrics. Our current pipeline uses only a limited amount of demand history, so adding richer lag and rolling-window features could help the models capture short-term momentum and repeated daily, weekly and even seasonal patterns more accurately. It would also be valuable to explore other machine learning approaches, such as LightGBM, sequence-based neural networks, or spatiotemporal models that can better represent temporal and geographic dependencies. Beyond improving the current regression setup, future work could examine related forecasting problems such as multi-step prediction, station-level bike availability, or joint pickup and drop-off modeling to support more proactive rebalancing decisions.

References

[1] Toronto Parking Authority. Bike Share Toronto 2025 Business Review and 2026 Annual Operating Plan. Published March 11, 2026. Accessed April 12, 2026. https://www.toronto.ca/legdocs/mmis/2026/pa/bgrd/backgroundfile-285022.pdf

[2] Lyft Urban Solutions. Toronto Bike Share Program. Accessed April 12, 2026. https://lyfturbansolutions.com/cities/toronto-bike-share

[3] CBC News. Bike Share Toronto adding more e-bikes, charging docks as demand goes up. Published March 12, 2026. Accessed April 12, 2026. https://www.cbc.ca/news/canada/toronto/bike-share-toronto-expand-service-2026-9.7125216

[4] Hansen M. Toronto to expand bike-share fleet as ebike demand surges. Canadian Cycling Magazine. Published March 13, 2026. Accessed April 12, 2026. https://cyclingmagazine.ca/sections/news/toronto-to-expand-bike-share-fleet-as-e-bike-demand-surges/

[5] Kaggle. Bike Sharing Demand. Accessed April 12, 2026. https://www.kaggle.com/competitions/bike-sharing-demand

[6] Hosseinpanahi D, Zadtootaghaj P, Lin J, Mohammadian A, Zou B. Machine Learning Models for Bike-Sharing Demand Forecasting. Future Transportation. 2026;6(1):26. https://doi.org/10.3390/futuretransp6010026

[7] Lee CKH, Leung EKH. Spatiotemporal analysis of bike-share demand using DTW-based clustering and predictive analytics. Transportation Research Part E: Logistics and Transportation Review. 2023;180:103361. https://doi.org/10.1016/j.tre.2023.103361

[8] Chen T, Guestrin C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016:785–794. https://doi.org/10.1145/2939672.2939785

[9] Grinsztajn L, Oyallon E, Varoquaux G. Why tree-based models still outperform deep learning on tabular data. Advances in Neural Information Processing Systems. 2022;35:507–520. https://doi.org/10.48550/arXiv.2207.08815

Appendix

All code, python notebooks, logs, trained models, and evaluation outputs used in this project are available in our public GitHub repository:

`https://github.com/abdueee/Toronto-Bikeshare-Forecasting`

AI Usage

AI was used mainly as a coding and learning aid throughout this project. In particular, it helped us get familiar with the syntax and usage of Python libraries used in the notebooks, including `pandas`, `numpy`, `matplotlib`, `requests`, `scikit-learn`, `xgboost`, and `PyTorch`. We used AI to better understand how to implement tasks such as API-based data collection with `requests`, K-means clustering with `sklearn.cluster.KMeans`, regression and evaluation with `LinearRegression`, `RandomForestRegressor`, `XGBRegressor`, and metrics such as MAE, RMSE, and R^2, as well as neural network components such as `StandardScaler`, `nn.Module`, `DataLoader`, and training-loop syntax in PyTorch. AI was also used to clarify plotting syntax for saving result figures and to help explain library functions and implementation details, but the project design, modeling choices, experiments, and final analysis were completed and verified by the group.

Contest for Information Sharing

As part of this course, we may share selected project materials (e.g., reports, presentation slides, and presentation recordings) on the course webpage as learning resources for future students. Additionally, we may use anonymized project information for internal statistical analysis of course outcomes. Please indicate your preferences below. *Your choices will not affect your grade in any way.*

Consent for Sharing Project Materials

Please keep the item you wish and remove the other one.

- The group members do **not** consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

Please keep the item you wish and remove the other one.

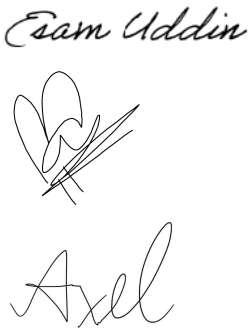
- The group members do **not** consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Optional Comments

We would like to thank Professor Ali Bereyhi and TAs for their guidance and feedback throughout the course and project. The structure of the milestones, assignments, lecture videos on YouTube, and support on Piazza made it much easier to stay on track throughout our project.

Group Identification

- Group number: 21
- Names of group members: Abdul Mohammed - 1012858481, Esam Uddin - 1012865384, Axel Tang - 1006832144
- Signature of group members:



- Date: Sunday, Apr 12, 2026